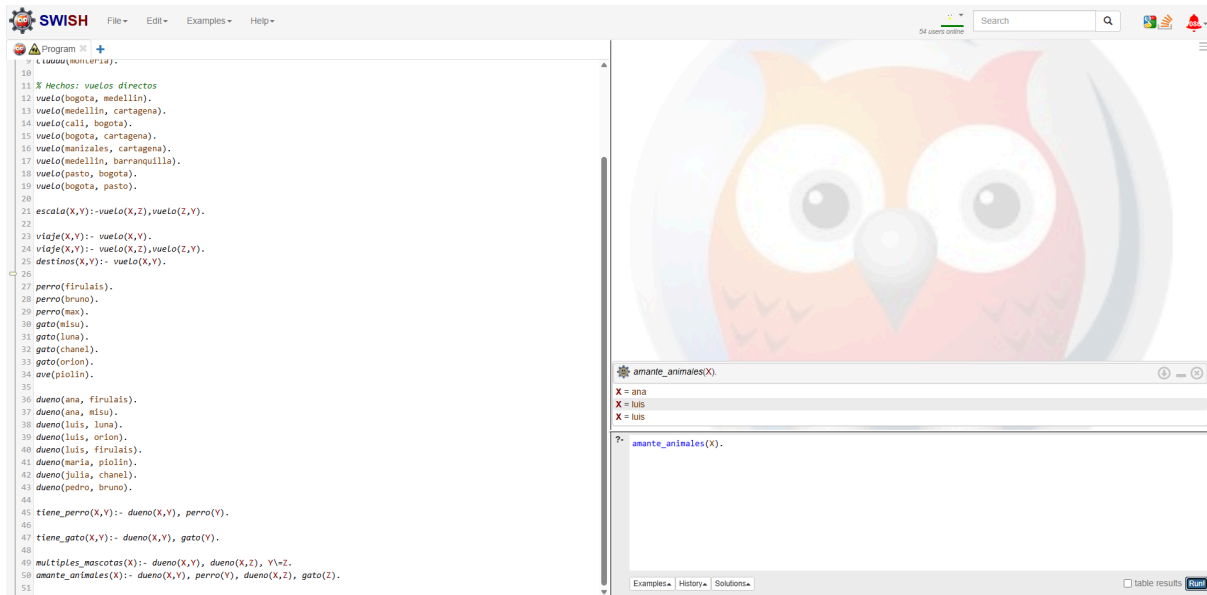
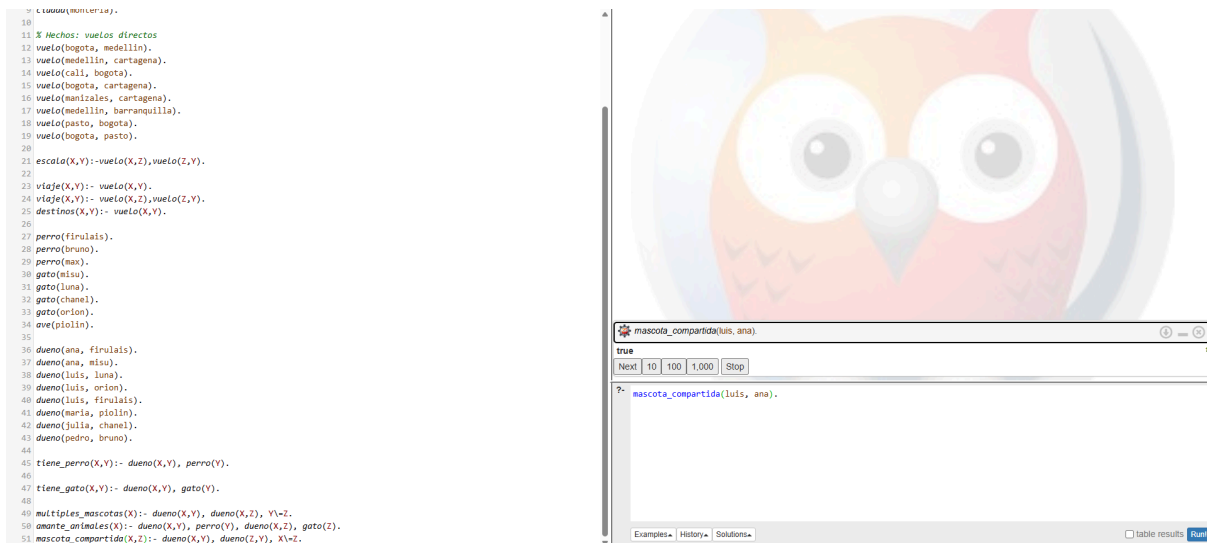


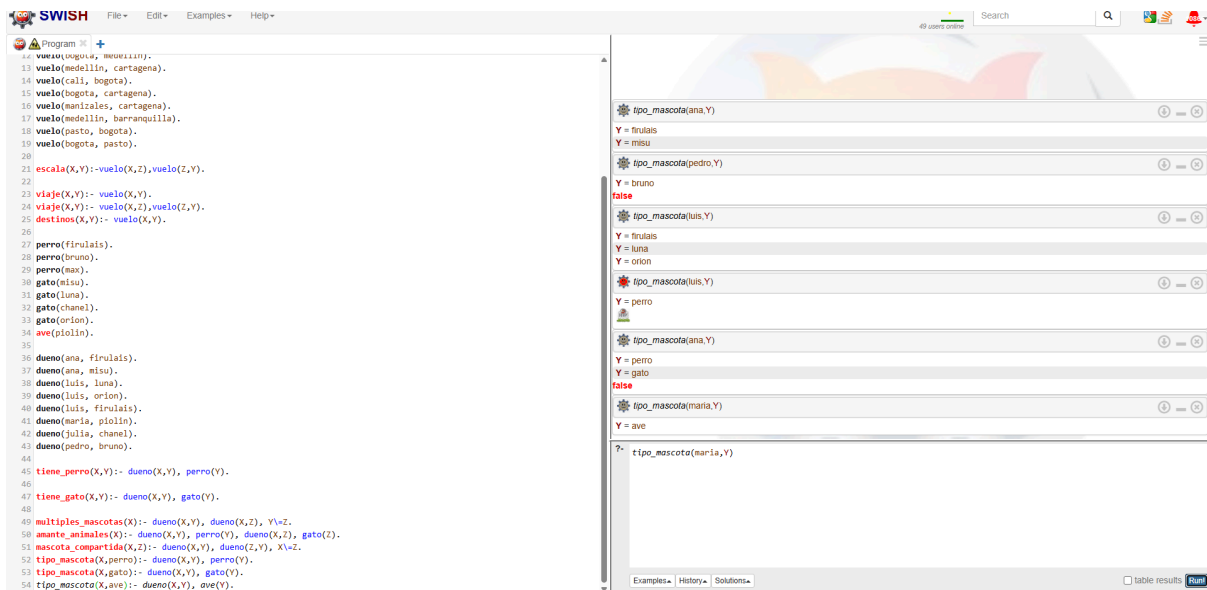
- Define una regla amante_animales para identificar dueños que tienen tanto perro como gato.



- Define una regla mascota_compartida que indique si dos personas comparten mascota.



- Define una regla tipo_mascota que asocie una persona con el tipo de mascota que tiene (perro, gato, ave, etc.).



Ejercicio 3 - Predicados con condiciones aritméticas

- Define una regla reprueba, que sea cierta si un estudiante tiene menor a 3.0.

```

7 reprueba(X,Y):- nota(X,Y), Y < 3.
8

```

- Define una regla aprueba, que sea cierta si un estudiante tiene nota mayor o igual a 3.0.

```

aprueba(X,Y):- nota(X,Y), Y >= 3.

```

- Define una regla rango, donde rango(X, Min, Max) sea cierto si la nota de un estudiante X está entre Min y Max.

```

rango(Y,X,Min,Max):- nota(Y,X), X >= Min, X <= Max.

```

- Define una regla clasificacion que asigne una categoría a cada estudiante según la nota:
 - 0.0 – 2.9 → reprobado
 - 3.0 – 3.9 → aprobado
 - 4.0 – 4.4 → notable
 - 4.5 – 5.0 → excelente

```
clasificacion(Est,C):- nota(Est,N), (N >= 0.0, N< 2.9 -> C = reprobado ; N >=3.0, N<3.9 -> C = aprobado;
  N >=4.0, N < 4.4 -> C = notable; N >=4.5, N < 5.0 -> C= excelente).
```

- Define la regla distancia((X1, Y1), (X2, Y2), D), que calcule la distancia euclidiana entre dos puntos

```
distancia(punto(X1, Y1), punto(X2, Y2), D) :- D is sqrt((X2 - X1)**2 + (Y2 - Y1)**2).
```

- Define la regla

distancia_total([P1, P2 | Resto], Total), que calcule la suma de distancias entre una lista puntos.

```
distancia_total([], 0).
distancia_total([_], 0).
distancia_total([P1, P2 | Resto], Total) :- distancia(P1, P2, D), distancia_total([P2 | Resto], DistanciaResto),
  Total is D + DistanciaResto.
```

Ejercicio 4 - Recursión simple

Define la regla fibonacci(X, N), donde X el el numero_x de la serie de fibonacci y F el valor correspondiente de la serie, uso de "is" para asignar valores a variables.

```
fibonacci(0, 0).
fibonacci(1, 1).
fibonacci(X, F) :- X > 1, X1 is X - 1, X2 is X - 2, fibonacci(X1, F1), fibonacci(X2, F2), F is F1 + F2.
```

Ejercicio 5 - 4 color problem

Faltaria definir la regla `adjacent` la cual valide la adyacencia valida.

```
1 color(red).
2 color(blue).
3 color(yellow).
4 color(green).
5 adjacent(X,Y):- color(X), color(Y), X\= Y.
6
7 map(A, B, C, D, E) :-
8     adjacent(A, B), adjacent(A, D), adjacent(A, E),
9     adjacent(B, C), adjacent(B, D), adjacent(B, E),
10    adjacent(C, D), adjacent(C, E),
11    adjacent(D, E).
```

Ejercicio 6 - Listas

- Escribe una regla que verifique si un elemento es miembro de una lista.
- Escribe una regla "longitud" que dada una lista, calcule su longitud.
- Escribe una regla "concatena" que dadas dos listas, las concatene en otra.
- Escribe una regla que permite obtener en una lista los N elementos de las serie de fibonacci, usa la regla predifina `append(L1,L2,L3)`, que concatena L1 y L2, en L3.
- Escribe una regla que determine el reverso de una lista.
- Escribe una regla que determine si una lista es palindroma

```
miembro(X, [X|_]).
miembro(X, [_|T]) :- miembro(X, T).

longitud([], 0).
longitud(_|T, L) :- longitud(T, L1), L is L1 + 1.

concatena([], L, L).
concatena([X|L1], L2, [X|L3]) :- concatena(L1, L2, L3).

reverso([], []).
reverso([H|T], R) :- reverso(T, RT), append(RT, [H], R).

palindromo(L) :- reverso(L, L).|
```